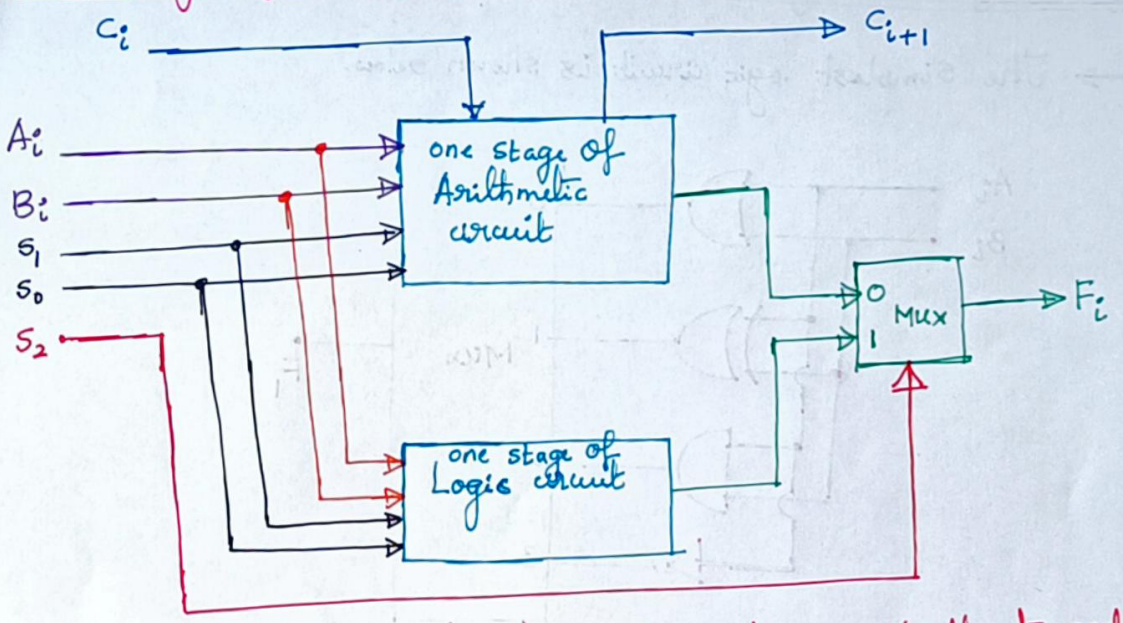
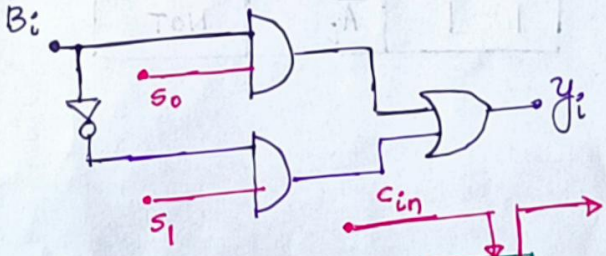
* Combining Logic and arithmetic circuits:



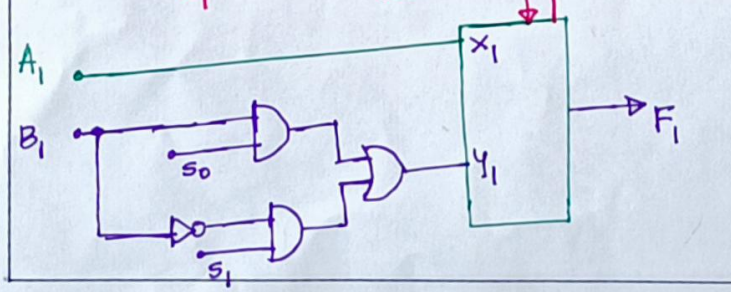
S_2: Mode - select Input: Distinguishes between Arithmetic and Logical operations

S_1, S_0: Function - select inputs: specify the particular arithmetic or logic operations to be performed.

* True/complement, one/zero circuit:



S_1	S_0	y_i
0	0	0
0	1	B_i
1	0	$\overline{B_i}$
1	1	1



* Logic operations in one stage of Arithmetic circuit

S_2	S_1	S_0	X_i	Y_i	$F_i = X_i \oplus Y_i$	Operation
1	0	0	A_i	0	A_i	Transfer A
1	0	1	A_i	B_i	$A_i \oplus B_i$	XOR
1	1	0	A_i	$\overline{B_i}$	$A_i \odot B_i$	Equivalence
1	1	1	A_i	1	$\overline{A_i}$	NOT

$A_i + B_i =$ 0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 1

$F_i = X_i \oplus Y_i = X_i \overline{Y_i} + Y_i \overline{X_i}$

1) $F_i = A_i \oplus 0 = A_i \cdot 1 + \overline{A_i} \cdot 0 = A_i \longrightarrow$ Transfer A

2) $F_i = A_i \oplus B_i = A_i \overline{B_i} + B_i \cdot \overline{A_i} = A_i \oplus B_i \longrightarrow$ XOR

3) $F_i = A_i \oplus \overline{B_i} = A_i \overline{\overline{B_i}} + \overline{B_i}\, \overline{A_i} = A_i B_i + \overline{A_i}\, \overline{B_i} = A_i \odot B_i \longrightarrow$ Equivalence

4) $F_i = A_i \oplus 1 = A_i \cdot 0 + \overline{A_i} \cdot 1 = \overline{A_i} \longrightarrow$ NOT

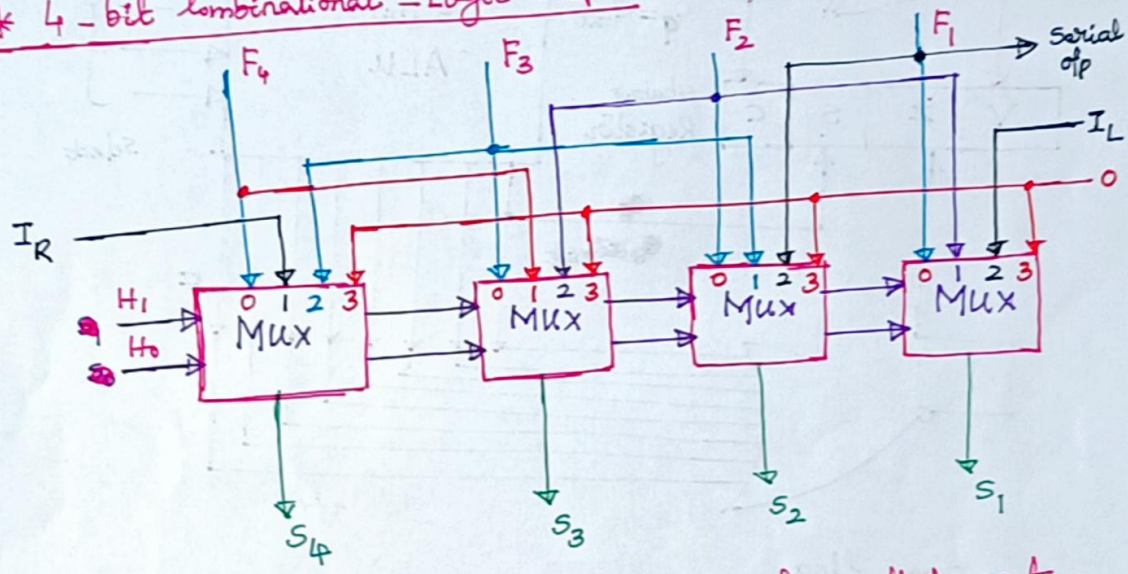
* ALU with Eight arithmetic operations and four logic operations

* Design of shifter:

↳ Transfers the output of the ALU onto the output bus.

↳ It may transfer the information directly without a shift, or it may shift the information to the Right or left.

* 4-bit combinational-Logic shifter:



H_1, H_0 → selection variables → Applied to all four multiplexers to select the type of operation in the shifter.

$H_1, H_0 = 00$ → No shifting take place, Data from F go directly to the S lines

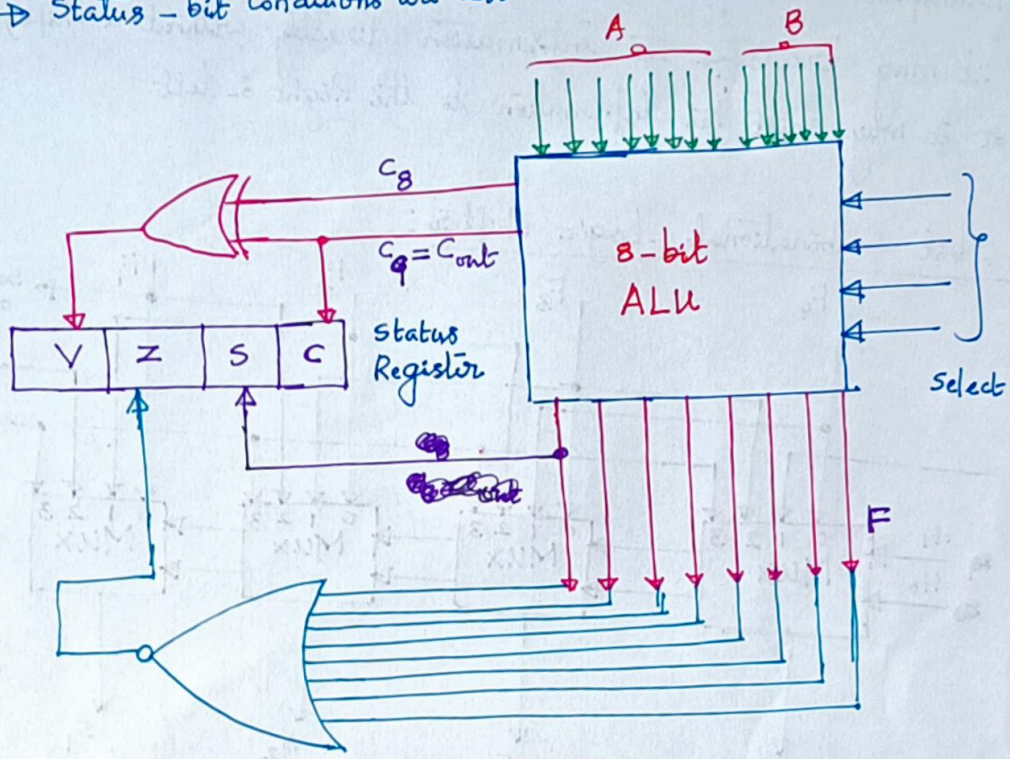
$H_1, H_0 = 01$ → Shift Right operation

$H_1, H_0 = 10$ → Shift Left operation

$H_1, H_0 = 11$ → the multiplexers select the inputs which are attached to '0' and as a consequence the S outputs are also equal to 0, blocking the transfer of Information from the ALU to the o/p bus.

* Status Register :

→ Status - bit conditions are also called as condition - code bits or flag bits.



C → carry flag

S → sign flag

Z → zero flag

V → overflow flag. (Signed numbers)

NOTE: 1) If Z=1, then A=B since A-B=0

If Z=0, then A≠B

2) overflow flag (V): V=1, XOR of two leftmost bits are =1

V=0, XOR of two leftmost bits are =0

Ex:

A = 10110110

B = 11001011

A+B = 1 10000001

C=1

S=1 ∵ MSB=1

Z=0 ∵ output ≠ 0

0 1

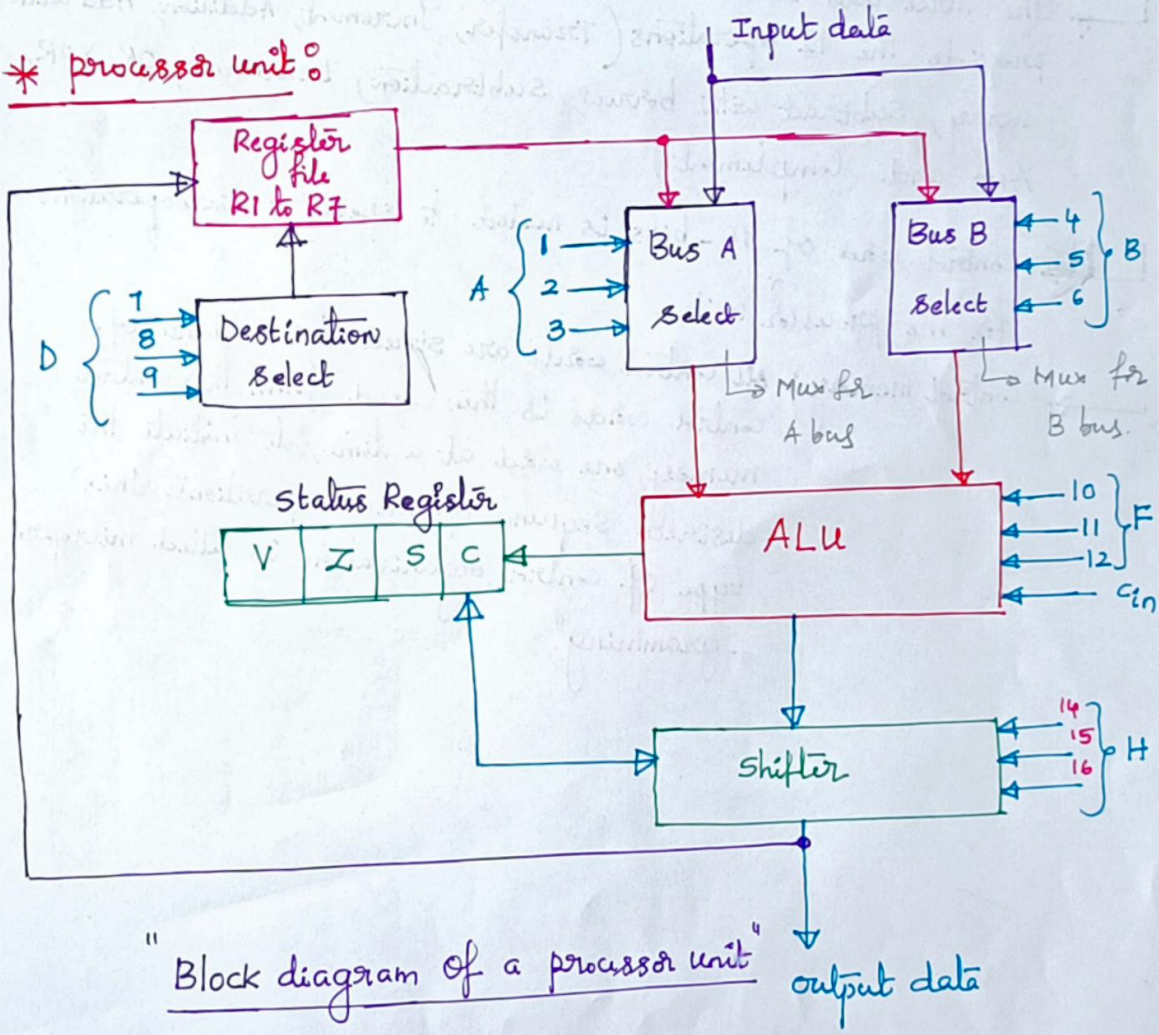
0 ⊕ 1 = 1

V=1

* Function table for transfer:

H_1	H_0	operation	Function
0	0	$S \leftarrow F$	Transfer F to S No shift
0	1	$S \leftarrow shr\,F$	shift Right F into S
1	0	$S \leftarrow shl\,F$	shift - Left F into S
1	1	$S \leftarrow 0$	Transfer 0's into S

* processor unit:



"Block diagram of a processor unit"

↳ The above diagram demonstrates, how the control variables select the microoperations in a processor unit

- There are 16 selection variables in the unit and their function is specified by a control word as shown below

1 2 3	4 5 6	7 8 9	10 11 12	13	14 15 16
A	B	D	F	C_{in}	H

→ control word

- The Registers (R1 to R7) are selected by A, B and D
- The three bits in the 'F' field, together with the input carry C_{in}, provide the 12 operations (Transfer, Increment, Addition, Add with carry, Subtract with borrow, Subtraction, Decrement, OR, XOR, AND and complement)
- Control word of 16-bits is needed to specify a microoperation for the processor unit.
- Control memory: All control words are stored, the sequence of control words is then read from the control memory, one word at a time, to initiate the desired sequence of micro operations. This type of control organization is called "microprogramming".